

Full Length Article

A collaborative audit scheme of IoT data integrity for fog computing

Xinfeng He^{a,b}, Qing Zhou^{a,b,*}^a School of Cyber Security and Computer, Hebei University, Baoding 071002, China^b Key Lab on High Trusted Information System of Hebei Province, Baoding 071002, China

ARTICLE INFO

Keywords:

Fog computing
 IoT data integrity
 Collaborative audit
 task allocation optimization, ADPSO

ABSTRACT

With the widespread application of Internet of Things (IoT) technologies, edge devices generate large volumes of distributed and latency-sensitive data. Fog computing has emerged as a key architecture in IoT environments due to its proximity to data sources and flexible computing and storage capabilities. However, current data integrity auditing schemes for fog computing, which rely on third-party auditors (TPAs) or blockchains, are limited in meeting IoT's high efficiency and low latency requirements. Specifically, TPAs suffer from audit latency that increases linearly with data scale—potentially reaching the minute level in large-scale IoT scenarios—and introduce single points of failure due to their centralized architecture, while blockchains incur substantial replication and consensus overhead that grows with the number of validators. To address these issues, we proposed a collaborative audit scheme of IoT data for fog computing (CAIFC). First, we constructed a decentralized auditing architecture that leveraged fog computing's low-latency advantage and employed a verifiable random function (VRF) to select main fog nodes (MFs) for parallel audit processing. A consistent hash ring mechanism was integrated to partition audit tasks logically and eliminate global consensus overheads. To support on-demand audit allocation, we designed a multi-dimensional load evaluation model that considered task security levels and latency requirements comprehensively and developed an adaptive discrete particle swarm optimization (ADPSO) algorithm to optimize audit allocation and resource utilization. Experimental results demonstrated that our scheme could reduce audit latency, enhance audit efficiency, and adapt to the needs of efficient data integrity audits in IoT environments.

1. Introduction

With the rapid development of Internet of Things (IoT) technologies, the number of IoT edge devices has been increasing exponentially (Ali et al., 2022). These devices are widely used in various domains, such as smart healthcare, intelligent transportation, and the Industrial Internet of Things (IIoT) (Chataut et al., 2023). They provide critical support for intelligent decision-making and automation control by continuously collecting, transmitting, and preprocessing data. However, the distributed deployment of IoT devices leads to highly dispersed data, which poses significant challenges to centralized management and efficient access. Additionally, IoT data are often characterized by large-scale and latency sensitivity. For example, geographically deployed sensors generate gigabytes of road data per second in intelligent transportation systems (Gong et al., 2023), requiring that the data can be transmitted in real-time to the cloud for storage and analysis. Data integrity is essential to IoT application reliability, and the aforementioned characteristics of IoT data impose more stringent requirements for ensuring their

integrity.

In IoT environments, fog computing has emerged as a key architecture that provides flexible computational support and decentralized service (Bonomi et al., 2012). Fog nodes, as middleware between IoT edge devices and the cloud, bring storage and computation capabilities closer to the data source, which can reduce latency and improve data processing efficiency. However, the inherent advantages of fog nodes have not been fully utilized by existing schemes, which remain limited in meeting IoT's high efficiency and low latency requirements. Traditional data integrity auditing schemes rely on third-party auditors (TPAs) for centralized verification (Wang et al., 2010; Zhao et al., 2019; Gao et al., 2021; Wen and Deng, 2024; Zhou et al., 2025; Tian et al., 2025). However, the audit latency increases linearly with the data scale as the TPA needs to process massive verification requests serially, e.g., the audit latency of IIoT data can reach the minute level. Furthermore, TPA becomes a single-point-of-failure bottleneck, which is incompatible with IoT's distributed architecture. In recent years, blockchain technology has been introduced into decentralized auditing schemes due to its

* Corresponding author.

E-mail address: zoeiy@stumail.hbu.edu.cn (Q. Zhou).<https://doi.org/10.1016/j.cose.2026.104935>

Received 10 April 2025; Received in revised form 9 December 2025; Accepted 20 April 2026

Available online 21 April 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

advantages of openness, transparency, and immutability (Han et al., 2022). However, blockchain-based schemes (Chen et al., 2018; Wei et al., 2020; Yuan et al., 2020; Chen et al., 2022; Liu et al., 2023) incur high storage and computational costs because of the global consensus mechanism and exhibit limited transaction throughput. These schemes struggle to satisfy the demand for efficient and low-latency auditing in IoT environments. Furthermore, most of the existing schemes adopt fixed or random audit task allocation and neglect task requirements such as security levels and latency requirements. They also overlook the load balancing of audit tasks, resulting in inefficient resource utilization. This inefficiency either overloads the audit nodes or leaves resources idle, significantly affecting the audit efficiency. Therefore, we need to design a distributed data integrity auditing scheme that is efficient, secure and adaptable to the IoT environment.

To address the aforementioned challenges, we proposed a collaborative audit scheme of IoT data integrity for fog computing (CAIFC). We fully considered the characteristics of IoT edge device data and replaced centralized TPA models with fog nodes to achieve decentralized auditing. Additionally, we employed task offloading technology to offload computationally intensive tasks from resource-constrained terminal devices to fog nodes with stronger processing capabilities. A multi-objective optimization strategy was utilized in the technology to achieve an on-demand allocation of audit tasks. The main contributions of this paper could be summarized as follows:

1) We proposed a dynamic management strategy for fog nodes and constructed a decentralized auditing architecture. The strategy combined the resource capacities of fog nodes and utilized a verifiable random function (VRF) to select the main fog nodes (MFs) of each group for parallel processing of audit requests. A two-layer hash mapping model with a consistent hash ring was integrated into the architecture. The model enabled logical partitioning of audit tasks and supported local task migration when MFs are dynamically joined/exited.

2) We constructed a multi-dimensional load evaluation model that considered the task security level and latency requirements comprehensively. Based on this model, we designed the adaptive discrete particle swarm optimization (ADPSO) algorithm to achieve optimal matching between audit tasks and fog node capabilities. This algorithm ensured efficient resource utilization while effectively addressing the load-balancing problem, and significantly improved the system performance and auditing efficiency.

2. Related works

Ateniese et al. (2007) first proposed a public audit model and designed the provable data possession (PDP) scheme. Random sampling and homomorphic verifiable tags were utilized to enable clients to verify whether a server held the original data correctly. Subsequently, they optimized the scheme by introducing precomputed verification tokens and pseudorandom functions, which improved verification efficiency and supported partial dynamic operations (2008). Based on the PDP scheme, Wang et al. (2010) introduced the third-party auditor (TPA) to alleviate the computational burden of clients and proposed a public audit scheme using BLS signatures and Merkle hash trees (MHT). The TPA had certain computational capabilities to launch auditing challenges and verify the data integrity. To address the issue of TPA's limited computing capabilities, Yang et al. (2012) proposed an optimization scheme that offloaded most of the auditing computation from TPA to the cloud. The computational overhead was significantly reduced by restricting the TPA to initiating auditing challenges and a subset of validation tasks. Xu et al. (2017) introduced an auditing task scheduling scheme based on dynamic resource allocation. They assessed the number of audit tasks assigned to validators, optimized task execution time, and allocated storage service providers' resources to minimize task loss rates and reduce storage resource waste. Researchers proposed a series of schemes for different audit requirements and application scenarios. Zhao et al. (2019) eliminated the need for users to maintain local index

tables by introducing a hierarchical authentication skip list. Additionally, they incorporated a timestamp receipt mechanism to ensure mutual supervision between users and servers. Yan et al. (2021) focused on the cloud storage of shared data and proposed an identity-based public auditing scheme. A randomization technique was employed to remap the association relationship between data uploaders and data blocks to enhance the anonymity of uploaders. To protect data privacy, Gao et al. (2021) designed a keyword-based auditing scheme, which utilized the relational authentication label (RAL) to safeguard sensitive information and allowed the TPA to verify files' integrity with access to a specific encrypted keyword. Wen et al. (2024) combined the sanitization signature with certificateless techniques to achieve dynamic auditing while protecting user privacy efficiently. Zhao et al. (2022) designed a certificateless public auditing scheme that supported designated verifiers. In this scheme, data owners could assign auditing tasks to specific TPAs as needed, preventing adversaries from launching malicious audit challenges against the cloud storage provider (CSP) and reducing unnecessary resource consumption. For multi-replica cloud storage, Zhou et al. (2025) proposed a certificate-based auditing scheme that adopted a leaves Merkle hash tree (LMHT) structure to achieve dynamic updates of multiple replicas and resist copy-summation attacks. Tian et al. (2025) proposed a multi-replica data auditing scheme based on the trusted execution environments (TEEs), which incorporated a random masking strategy and a dual authentication mechanism to reduce user's computational burden effectively and improve system security significantly.

The aforementioned research schemes relied on TPA to conduct centralized auditing. These schemes were susceptible to single-point failures, and the audit latency increased linearly with the growth of data. In addition, TPA might launch collusion attacks with CSP, making it difficult to meet auditing requirements in distributed environments. Blockchain was widely used in cloud auditing schemes for its advantages of decentralization, immutability, and transparency. In a blockchain network, each server or client could act as a node, participate in a consensus mechanism, and maintain a distributed ledger for data integrity auditing. This decentralized structure reduced the risk of single-point failures significantly. In recent years, researchers have proposed various blockchain-based cloud auditing schemes. Chen et al. (2018) designed a random blockchain-based data integrity verification scheme for IoT environments. The scheme reduced the computational overhead by randomly selecting collaborative nodes and a lightweight mining process. Wei et al. (2020) made full use of the mobile agent technology to deploy a distributed virtual machine agent model. They also combined MHT with smart contracts to monitor data changes so that the blockchain and the agent VMs could jointly verify data integrity. Yuan et al. (2020) designed a blockchain-based public auditing and data deduplication scheme, which implemented an automated reward and punishment mechanism through smart contracts to punish CSPs for dishonest behaviors and compensate DO for impaired data. Chen et al. (2022) proposed a blockchain-based random auditor committee scheme, which employed a verifiable random function to select a third-party auditor committee (TPAC). The leader of TPAs (LTPA) initiated audit challenges, and TPAC audited data integrity using tamper-proof evidence on the blockchain, which effectively defended against collusion attacks. Liu et al. (2023) considered the impact of the audit cycle and proposed an auditing scheme that integrated blockchain with document prediction technology. Their scheme utilized lightweight signatures and smart contracts to support user revocation and file prediction, ensuring both data security and audit efficiency. Hu et al. (2024) developed a scheme that supported editable blockchain functionality. Their approach enabled user identity tracking by incorporating chameleon hashing and bilinear mapping, making blockchain audit more flexible and efficient.

Although blockchain technology has demonstrated many advantages in auditing, it still has some limitations. The aforementioned schemes often suffered from high storage and computational costs caused by

global consensus mechanisms, heavy client audit burdens, and limited transaction throughput. To meet the demand for high-frequency and real-time data audits in IoT environments, researchers have begun to explore the application of fog computing in cloud architectures. Tian et al. (2019) proposed a public audit scheme in a cloud-fog computing scenario, which optimized the integrity verification process for multi-source IoT data. Their approach leveraged the preprocessing capability of fog nodes and the storage resources of cloud servers, employing both a label transformation strategy and a zero-knowledge proof mechanism to enhance efficiency. Sellami et al. (2024) designed a distributed audit scheme for fog computing architecture using the short integer solution problem (SIS). The scheme allowed clients to distribute data blocks across multiple servers in different regions, while the identity-based signatures (IBS) technique ensured data block homology verification. Liu et al. (2024) proposed a dynamic audit scheme based on code storage, which was particularly suited for cloud-fog collaborative industrial IoT data storage. The scheme encoded data files using exact repair regenerating codes and generated authentication tags for each encoded data segment through ZSS signatures. Bian et al. (2022) proposed a data-blindness-based audit scheme, which introduced a fog computing layer in the data transmission layer to optimize the data transmission path and enhanced the security of auditing by using the blind factor technique. Chang et al. (2023) developed a certificateless dynamic data sharing and auditing scheme, which eliminated key escrow vulnerabilities and supported dynamic updates. Gu et al. (2024) focused on vehicular ad-hoc networks (VANETs) in cloud-fog computing environments and proposed a blockchain-based deduplication and distributed auditing scheme. They constructed a distributed auditing model that utilized blockchain technology to ensure data integrity during uploads and auditing processes. Additionally, they introduced a lightweight dual validator architecture, in which roadside units (RSUs) and fog nodes collaboratively executed auditing tasks.

Existing schemes had provided numerous solutions for data audit. However, most of these solutions relied on TPA or blockchain technology, both of which had exhibited inherent limitations. Additionally, they adopted fixed or random audit task allocation without considering the types of audit requests and task scheduling requirements, ignoring both the security level and latency requirements of audit tasks as well as the load balancing of the tasks. These schemes struggled to accommodate real-time, high-frequency, and multi-source data auditing in IoT environments. This paper aimed to overcome these limitations and provided an efficient and secure integrity audit scheme for IoT data stored in the cloud.

3. Preliminaries

3.1. Verifiable random function

A verifiable random function (VRF) was proposed by Micali et al. (1999), which allows the user to map inputs to verifiable

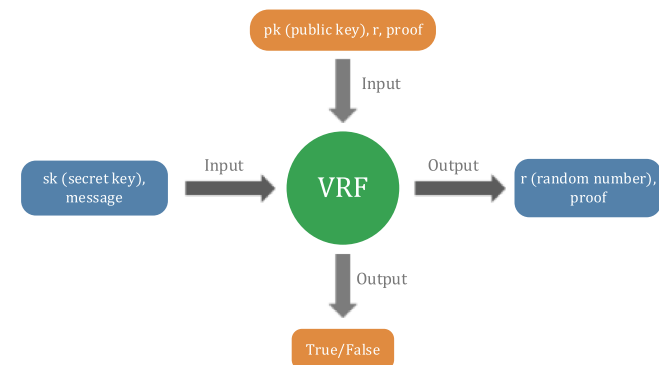


Fig. 1. The workflow of VRF.

pseudo-random outputs. The workflow is shown in Fig. 1, a verifier generates a private key sk and a public key pk using a key generation algorithm. For a given input m and sk , the VRF outputs a pseudo-random value $r = \text{VRF.Hash}(sk, seed) = e(g, g)^{\frac{1}{sk+seed}}$ together with a cryptographic proof $proof = \text{VRF.Proof}(sk, seed) = g^{\frac{1}{sk+seed}}$. Using the public key pk , any verifier can check the validity $e(g^{seed}pk, proof) \stackrel{?}{=} e(g, g)$ efficiently based on the provided proof. A verifiable random function satisfies three properties:

- Verifiability: Anyone with a public key can verify that the random numbers generated by the VRF are valid.
- Randomness: The output of the VRF is pseudo-random for different inputs, and the output can be guessed with negligible probability of success.
- Uniqueness: Given the same input and secret key, the output of the VRF remains unique.

3.2. Consistent hash ring

A consistent hash ring is a widely used load-balancing algorithm in distributed systems (Karger et al., 1997). It achieves an efficient allocation of resources and fast routing of user requests by mapping data and nodes onto a virtual ring. Assuming that the value space of the hash function ranges from 0 to $2^{32}-1$, all input values are mapped into this range. The consistent hash ring has high stability: when a node is added or removed from the system, only the data associated with that node and its successor are affected, while the data of other nodes remain unchanged. To prevent uneven data distribution, this paper adopts a consistent hash ring with virtual nodes, as shown in Fig. 2. Multiple virtual nodes are generated and mapped onto the hash ring by computing multiple hash values for each physical node.

3.3. PSO algorithm

Particle swarm optimization (PSO) is an optimization technique based on swarm intelligence (Kennedy et al., 1995). The algorithm simulates the social behavior of bird flock foraging, and searches for the optimal solution in the solution space through the mechanism of collaboration and information sharing between individuals. PSO features a simple structure and ease of parallel implementation, making it suitable for multi-objective optimization, scheduling optimization, and other related problems. However, PSO suffers from premature convergence and stagnation when dealing with complex optimization problems. To address these issues, this paper adopts the adaptive particle swarm optimization (APSO) (Singh et al., 2024), with the workflow

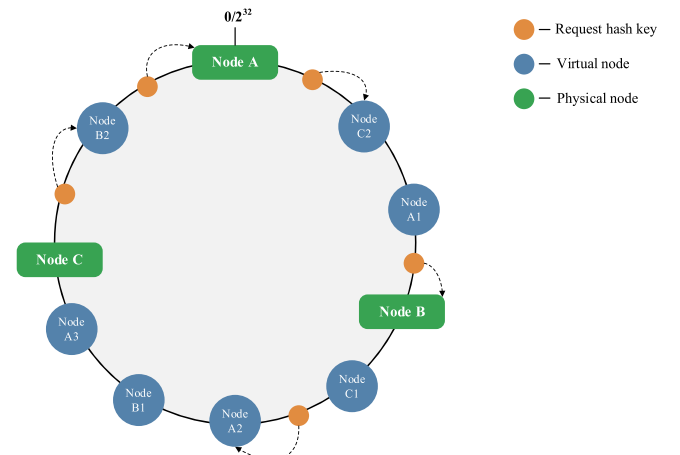


Fig. 2. Consistent hash ring.

shown in Fig. 3. It can balance the ability of global exploration and local exploitation by dynamically adjusting the inertia weights and the cognitive factor, preventing the algorithm from getting trapped in sub-optimal solutions.

4. Decentralized audit architecture and optimization mechanism for fog computing

4.1. Decentralized audit architecture

The audit architecture of this scheme was shown in Fig. 4, which divided the fog nodes into two entities: main fog nodes (MFs) and ordinary fog nodes (OFs). The architecture included four entities: data owner (DO), cloud storage provider (CSP), MF, and OF. The functional descriptions of each entity were as follows:

1. **DO:** As the holder of IoT edge device data, DO was responsible for generating encrypted files and their corresponding authentication tags and uploading this information to the CSP for storage. To ensure data integrity and security, DO could initiate audit requests to the fog nodes on demand;
2. **CSP:** CSP provided rich storage and computational resources to store the data uploaded by the DO. Upon receiving a valid audit challenge, the CSP generated an integrity proof and returned it to the auditor for verification;
3. **MF:** MF, as the core node of the fog nodes, undertook key functions such as task scheduling, resource management, and batch result feedback. MF mapped onto the hash ring by a consistent hash function and stored the resource and geo-location information of OFs in the group. To enhance audit efficiency and resource utilization, the MF executed the ADPSO algorithm upon receiving an audit request from the DO and assigned the audit task to the most suitable OF. Additionally, the MF aggregated batch audit results and provided feedback to the DO, ensuring transparency and traceability throughout the audit process.
4. **OF:** As the executor of data integrity verification, the OF initiated audit challenges to the CSP and verified the returned integrity proofs. The verification results were then reported to either the MF or the DO, ensuring the efficient execution of the audit process.

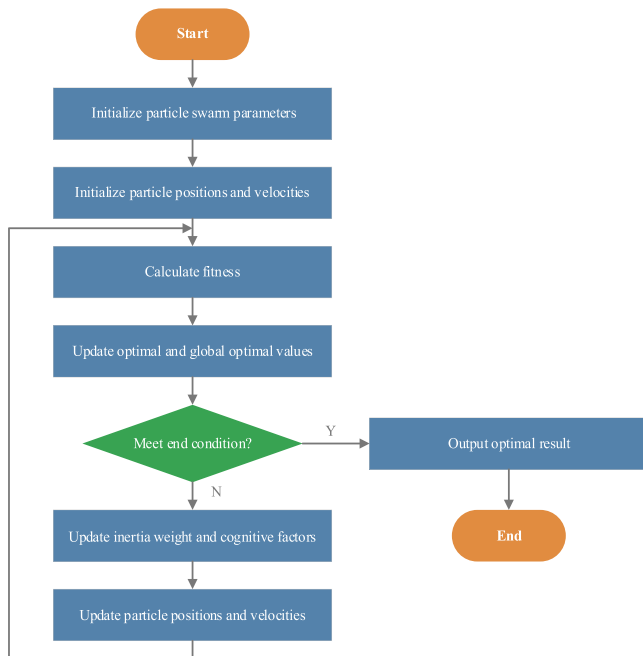


Fig. 3. The workflow of APSO algorithm.

The DO uploaded the encrypted file and corresponding authentication tags to the CSP for storage, and initiated an audit request to a specific MF on demand through the consistent hash ring mechanism. Upon receiving the request, the MF utilized the ADPSO algorithm to assign the audit task to the most suitable OF. The OF then initiated an audit challenge to the CSP, and the CSP responded to the challenge and generated a proof of integrity to be returned to the OF. The OF performed a verification of the proof and returned the verification results to the MF or directly to the DO.

4.2. Dynamic management strategy for fog nodes based on VRF and consistent hash ring

IoT edge device data typically exhibited high-frequency and real-time characteristics. The traditional TPA-based centralized auditing model was prone to issues such as single point of failure, high auditing latency, and privacy leakage when applied to IoT edge device data. Although the decentralized blockchain-based auditing model mitigated the single-point failure problem, it introduced significant drawbacks, including high storage overhead, excessive computational costs, and limited transaction throughput. Fog nodes at the edge layer can reduce auditing latency and improve data processing efficiency due to their storage and computing capabilities closer to the data source. However, their limited computational power made them susceptible to excessive workloads. When handling high-frequency audit requests, they were prone to overloading of computation, leading to saturated congestion of fog nodes and loss of audit requests. To address the above challenges, we designed a dynamic management strategy tailored for the edge layer to replace the traditional TPA-based centralized auditing model and formed a decentralized auditing architecture. Specifically, a verifiable random function (VRF) was introduced to enable secure, fair, and permissionless MF election without relying on a centralized authority. This design ensured the unpredictability and public verifiability of the election process. A consistent hash ring mechanism was employed to provide stable and efficient task-to-node mapping under dynamic fog group. The strategy constructed a two-layer hash mapping model to map the key values of data blocks and IP addresses of MFs to the virtual ring. The implementation of this strategy was divided into two key phases: election and mapping.

4.2.1. Election phase

As shown in Fig. 5, we first divided the fog nodes into groups according to the geographic location, assuming three groups in this scenario. During the initial system deployment, each fog node provided its initial resource profile, including computational, communication, and storage capabilities, as well as its geographic location. We then employed a VRF combined with the resource capacities of the fog nodes to select the MF_i fairly. The elected MF_i subsequently took over management and coordination responsibilities, while the remaining nodes within each group were designated as OF to enable parallel execution of the audit. Once the system became operational with the elected MF_i, a new fog node joined the system by registering with the MF_i of its geographically proximate group. The new node submitted its resource profile and was admitted as an OF node. Thereafter, its resource capabilities were subject to dynamic evaluation within the group, ensuring that subsequent elections were based on up-to-date capability information. The detailed workflow was as follows:

- 1) **KeyGen** (1^k) $\rightarrow$ (fsk_i, fpk_i). Each fog node executed a key generation algorithm to generate its private key fsk_i and public key fpk_i .
- 2) **FogGen** (seed, fsk_i , $snode_i$) $\rightarrow$ (r , proof). DO selected the random seed and broadcast it to all fog nodes within the group. Each fog node computed $snode_i = w_1 * c_i + w_2 * b_i + w_3 * s_i$, where c_i denoted the normalized computational resource, b_i represented the normalized communication resource, and s_i indicated the normalized storage resources. The variable i belonged to the range $[1, N]$, where N

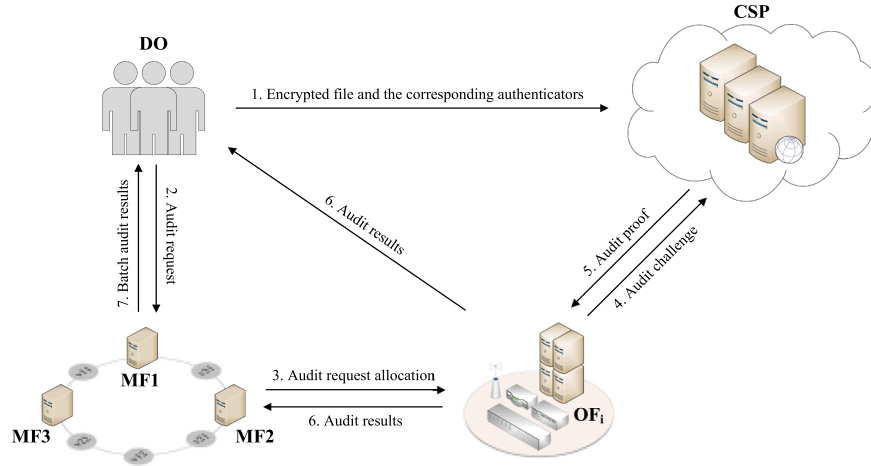


Fig. 4. Audit architecture.

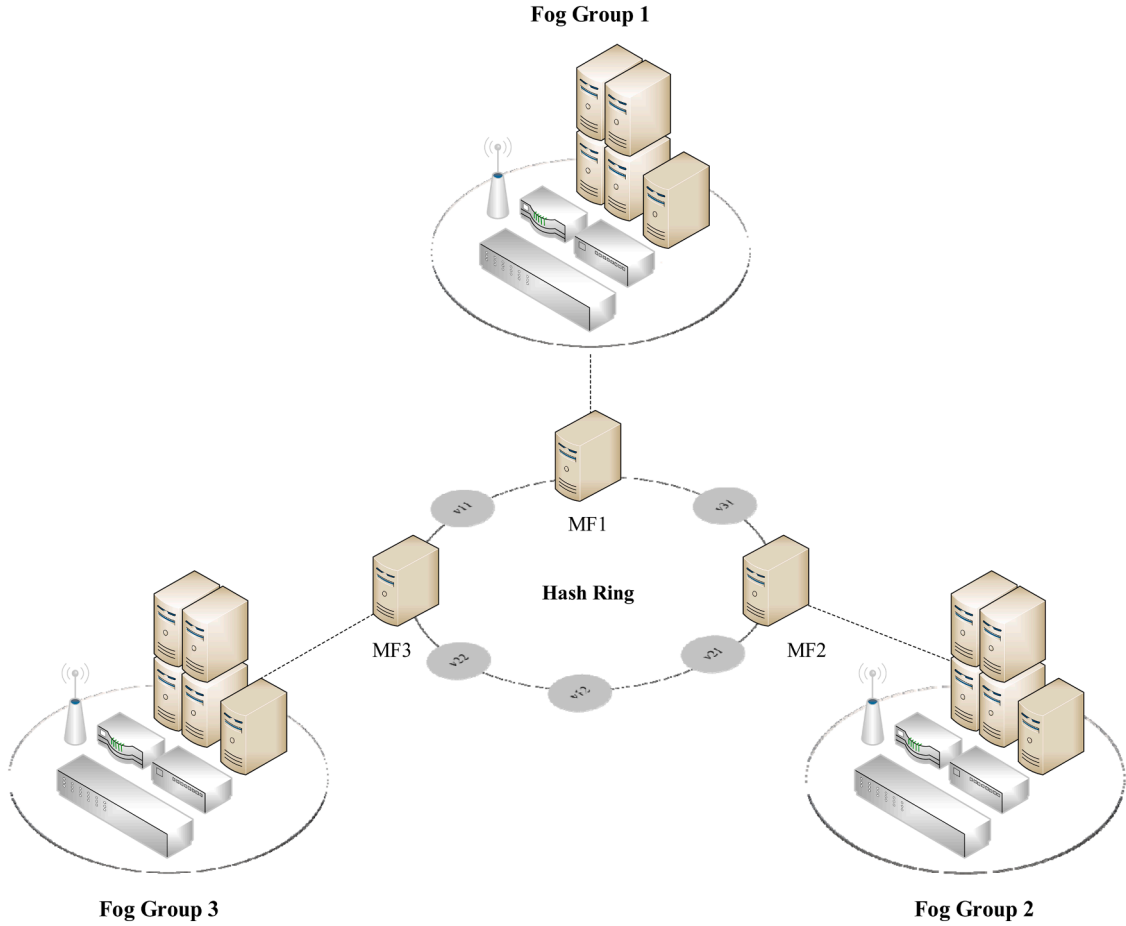


Fig. 5. The group of fog nodes.

denoted the number of fog nodes. The weights $w_1 \sim w_3$ determined the resource contribution, satisfying the condition $w_1 + w_2 + w_3 = 1$. The value of $snode_i$ was periodically updated to ensure that the MF election relied on the most recent capability information. To improve efficiency, only fog nodes with $snode_i$ values exceeding a certain threshold were eligible to participate. These nodes took the random seed and fsk_i as inputs to compute a random number r and a zero-

knowledge proof, where $r = VRF_Hash(fski, seed) = e(g, g)^{\frac{1}{fski+seed}}$, $proof = VRF_Proof(fski, seed) = g^{\frac{1}{fski+seed}}$.

- 3) VerifyMF($fpki, seed, r, proof, snode_i$) $\rightarrow$ (MFID). Each fog node verified the computation by inputting its respective $fpki$, $seed$, r , and $proof$. At this stage, the fog nodes acted as verifiers to validate whether [equation \(1\)](#) held.

$$e(g^{seed} fpki, proof) \stackrel{?}{=} e(g, g) \quad (1)$$

If the equation was satisfied, the verification was considered successful. The fog node with the highest $snode_i$ value within the group was selected as the MF_i. The system assigned it a unique identifier MFID, while the remaining nodes in the group were defined as OF nodes. The MF_i recorded and stored the geographic location as well as the resource information of each OF in the group. An MF could revert to the OF role either voluntarily (e.g., after completing its service period) or involuntarily due to failure or performance degradation, thereby triggering a new election. Conversely, any qualified OF could be promoted to MF through the same election process.

4.2.2. mapping phase

We designed a two-layer hash mapping model to build an efficient, decentralized, and load-balanced distributed audit architecture. This model was based on the consistent hash ring mechanism to manage the MF node dynamically and map audit tasks efficiently. The specific implementation scheme of the mapping phase was as follows:

- 1) Hash mapping: The entire hash space was organized into a virtual hash ring, assuming that the value space of the hash function was from 0 to $2^{128} - 1$. The identity MFID and IP address of each MF node were computed by the hash function to calculate $H(MFID||IP)$, producing a 128-bit hash value that was mapped to the corresponding position on the hash ring to form logical partitions. The key value (DataBlockID) of the audit task could also be calculated by the same hash function to calculate $H(DataBlockID)$ and the nearest MF node in the clockwise direction was selected for task assignment. As shown in Fig. 6, the system was divided into three groups, each containing one MF node (MF₁, MF₂, MF₃), which were mapped to different locations on the hash ring. They formed a decentralized auditing architecture that avoids bottlenecks in centralized auditing models effectively.
- 2) Virtual Node Mechanism: To address task distribution imbalance, we introduced a virtual node mechanism. Each MF_i node generated several virtual nodes V_{ij} on the hash ring, and the number of these nodes was dynamically adjusted based on the resources and computational capability of the MF node. Nodes with greater computational capability generated more virtual nodes and handled

a larger proportion of tasks. As shown in Fig. 6, MF₁ generated virtual nodes $\{v_{11}, v_{12}\}$, MF₂ generated virtual nodes $\{v_{21}, v_{22}\}$, and MF₃ generated a virtual node $\{v_{31}\}$.

- 3) Partial task migration: When MF nodes were dynamically added or withdrawn, the system only needed to migrate the auditing tasks covered by the affected virtual nodes without a global reconfiguration, thereby improving the system's scalability and stability. When a new MF node was added, it was inserted at the appropriate position on the hash ring according to its hash value, and corresponding virtual nodes were created to take over tasks from the neighboring regions, achieving elastic system expansion. When an MF node failed or needed to exit, tasks originally mapped to that node were migrated to the neighboring MF nodes in a clockwise direction. For example, if MF₂ failed, the tasks handled by its virtual nodes v_{21} and v_{22} were reassigned to neighboring virtual nodes (e.g., v_{31} or v_{11}).

4.3. ADPSO-based optimization strategy for on-demand allocation of audit tasks

Traditional audit schemes mostly adopted fixed or random allocation of audit requests, and these allocation modes had the following limitations: firstly, they failed to fully consider the security level and latency requirements of the audit tasks and also lacked the consideration of load balancing and scheduling of the tasks, leading to low utilization of audit resources and affected the auditing efficiency. Secondly, they might have the situation of task malicious losses or audit verification failures, which reduced the reliability and security of the system. To enable on-demand task allocation and assign audit tasks to the most suitable fog nodes dynamically, the algorithm needed to address discrete decision-making and multi-objective optimization simultaneously. In this context, Particle swarm optimization (PSO) algorithms, known for their simple structure and ease of parallel implementation, had been widely used in multi-objective optimization and scheduling problems. However, PSO was prone to issues such as premature convergence or local optimal solutions when dealing with complex optimization problems. To address these challenges, we designed a task allocation optimization strategy. Firstly, we constructed a multi-dimensional load evaluation model that comprehensively considered various factors, including task security levels, audit latency demands, and resource status. Secondly, we designed an ADPSO algorithm, which avoided premature convergence and ensured the optimal allocation of the audit requests by adjusting the particle swarm search strategy adaptively.

4.3.1. Multi-dimensional load evaluation model

Audit files were classified into four types based on their priority, as shown in Table 1. The resource requirement of a file was expressed as $File_i = (ra_i, rc_i, rb_i, rs_i)$, where ra_i represented the normalized file type, rc_i denoted the normalized computational resource requirement, rb_i indicated the normalized communication resource requirement, rs_i referred to the normalized storage resource requirement. Here, $i \in [1, m]$, where m was the number of files. DO calculated the file priority score based on its resource requirements using the formula: $sf_i = w_a * ra_i + w_c * rc_i + w_b * rb_i + w_s * rs_i$, where $w_a \sim w_s$ represented the weight coefficients of different resource requirements, satisfying $w_a + w_c + w_b + w_s = 1$.

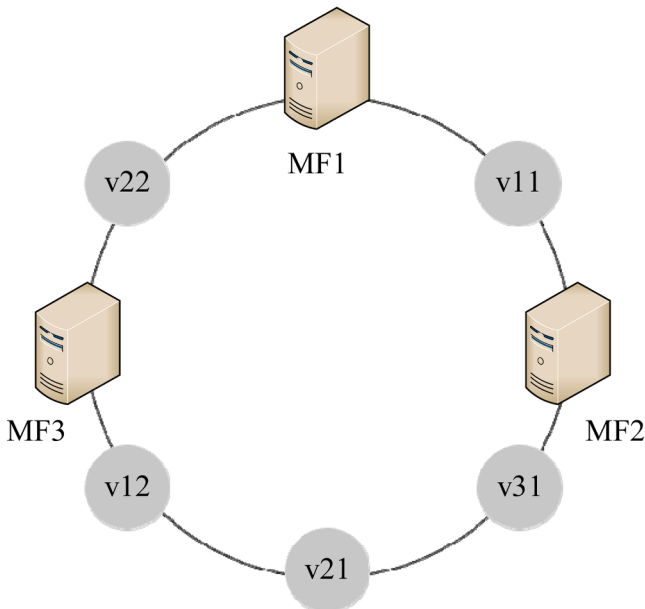


Fig. 6. MF mapping.

Table 1
Classification of audit file types.

Type code	Latency requirement	Security level
00	Delay-tolerant	Low transactionality
01	Delay-tolerant	High transactionality
10	Delay-sensitive	Low transactionality
11	Delay-sensitive	High transactionality

4.3.2. ADPSO algorithm

I. Particle representation

Each particle represented a possible task allocation solution, and its dimension was determined by the number of audit tasks (M) and the number of OFs (N). Given an audit task set $\{T_1, T_2, \dots, T_M\}$ and a node set $\{O_1, O_2, \dots, O_N\}$, the position of each particle was expressed as a vector $p = [p_1, p_2, \dots, p_M]$. Each element p_i in the vector denoted the specific OF node assigned to task T_i , where $p_i \in [1, N]$. For example, the particle $p = [1, 3, 2]$ indicated that task T_1 was assigned to fog node O_1 , T_2 was assigned to fog node O_3 , and task T_3 was assigned to fog node O_2 .

II. Fitness function

The optimization objectives of auditing task allocation included minimizing latency, maximizing edge-layer contribution, and optimizing load balancing among fog nodes. MF employed the ADPSO algorithm to allocate the resource capacity of OF nodes based on the optimization objectives of task allocation. Therefore, the fitness function needed to combine multiple objective functions: latency function, edge-layer contribution function, and load balancing function. Assume that f_{delay} , $f_{\text{contribution}}$, and f_{balance} denoted the objective functions of latency, edge-layer contribution, and load balancing respectively.

1. Latency Optimization

Latency was a critical metric in IoT environments, and latency-sensitive tasks needed to be assigned to OF nodes with high computational capacity and high bandwidth to minimize network and processing delays. In $f_{\text{delay}}(p)$ function (2), the transmission latency from task T_i to OF node O_j was defined as d_{ij} , and the processing time of T_i on O_j was denoted as p_{ij} . The variable x_{ij} indicated whether T_i was assigned to O_j ($x_{ij} = 1$) or not ($x_{ij} = 0$).

$$f_{\text{delay}}(p) = \sum_{i=1}^M \sum_{j=1}^N (d_{ij} + p_{ij}) * x_{ij} \quad (2)$$

$$\text{where } d_{ij} = \frac{r_{bi}}{b_j}, p_{ij} = \frac{r_{ci}}{c_j}$$

2. Edge-Layer Contribution Optimization

The edge-layer contribution (3) was influenced by the computational, communication, and storage resource utilization of each fog node. High-priority tasks had a greater impact on the contribution, and the objective was to maximize this contribution. The weight coefficients $\{v_c, v_b, v_s\}$ were used to balance different resource factors. The completion rate of tasks on node O_j was represented as F_j , while u_j denoted the resource utilization of O_j . η_{ij} indicated whether node O_j successfully completed a task ($\eta_{ij} = 1$) or not ($\eta_{ij} = 0$).

$$f_{\text{contribution}}(p) = \sum_{j=1}^N F_j \cdot u_j \quad (3)$$

$$\text{where } F_j = \frac{\sum_{i=1}^M r_{ai} \cdot x_{ij} \cdot \eta_{ij}}{\sum_{i=1}^M x_{ij}}, u_j = v_c \frac{\sum_{i=1}^M r_{ci} \cdot x_{ij}}{c_j} + v_b \frac{\sum_{i=1}^M r_{bi} \cdot x_{ij}}{b_j} + v_s \frac{\sum_{i=1}^M r_{si} \cdot x_{ij}}{s_j}$$

3. Load Balancing Optimization

The scheme aimed to ensure that the resource utilization across OF nodes in the same group remained balanced, preventing some nodes from being overloaded while others remained idle. As shown in function (4), the degree of balanced resource utilization was measured by the standard deviation of resource utilization $\sigma(u_j)$ among the nodes, with $\bar{u}$ representing the average resource utilization of all OF nodes within the group.

$$f_{\text{balance}}(p) = 1 - \frac{\sigma(u_j)}{\max(u_j)} \quad (4)$$

$$\text{where } \sigma(u_j) = \sqrt{\frac{1}{N} \sum_{j=1}^N (u_j - \bar{u})^2}, \bar{u} = \frac{1}{N} \sum_{j=1}^N u_j$$

In summary, the fitness function (5) was formulated as follows:

$$\text{fitness}(p) = r_1 * \frac{1}{f_{\text{delay}}(p)} + r_2 * f_{\text{contribution}}(p) + r_3 * f_{\text{balance}}(p) \quad (5)$$

Where r_1 , r_2 , and r_3 were weight coefficients that controlled the trade-offs among latency, edge-layer contribution, and load balancing.

III. ADPSO algorithm steps

1. Initialization of algorithm parameters

MF initialized the relevant parameters of the algorithm: (1) the inertia weighting factor w , which controlled the velocity of the particles and was usually set within the range $[0.4, 0.9]$; (2) the cognitive factor c_1 and the social factor c_2 , which balanced the trade-off between global exploration and local exploitation; (3) The initial positions and velocities of the particles were randomly assigned by MF, where the velocity v_i took an integer value in the range $[-v_{\text{max}}, v_{\text{max}}]$; (4) The global best solution g_{best} and individual best solution p_{best} were initialized.

2. Fitness calculation

The fitness function $\text{fitness}(p)$ was calculated based on the current position of the particle.

3. Updating best solutions

If the fitness of a particle was better than the current p_{best} , then the p_{best} was updated. And if the fitness of a particle was better than g_{best} , then the g_{best} was updated.

4. Termination conditions and adaptive parameter adjustment

During the iteration process, MF determined whether the algorithm met one of the following termination conditions: (1) the maximum number of iterations I_{max} was reached; (2) the fitness function reached a preset threshold; (3) The fitness exhibited no significant change over multiple iterations. If the termination condition was not met, the parameters w , c_1 and c_2 were updated adaptively based on the following strategies:

➤ When high loads were detected on certain nodes or network delays increased, the inertia weight w was reduced to accelerate convergence and promptly adjust task assignments. When the system remained stable, a higher inertia weight was maintained to continue exploring possible optimization solutions. As shown in Eq. (6), w_{min} and w_{max} denoted the minimum and maximum inertia weights respectively, and $s = 13$ was the speed control coefficient. Additionally, we designed the $f(p)$ function due to a multi-objective optimization, where r_4 denoted the weight of $f(p)$ and could be adjusted adaptively.

$$w^i = \left(w_{\text{min}} + \frac{w_{\text{max}} - w_{\text{min}}}{1 + e^{\frac{2s}{I_{\text{max}}} (i - 0.5)}} \right) \cdot (1 - r_4 \cdot f(p)) \quad (6)$$

$$\text{where } f(p) = r_1 \cdot f_{\text{delay}}(p) - r_2 \cdot f_{\text{contribution}}(p) - r_3 \cdot f_{\text{balance}}(p)$$

➤ The parameters c_1 and c_2 regulated the speed at which particles moved toward p_{best} and g_{best} . In the early stage of the algorithm, a higher c_1 and a lower c_2 were assigned to encourage particles to explore the global solution space. In the later stage of the algorithm, c_1 was gradually decreased while c_2 was increased to accelerate convergence speed and enhance group guidance. The specific adjustment formulas (7) and (8) were given as follows:

$$c_1^i = c_{\text{min}} + \frac{(c_{\text{max}} - c_{\text{min}}) \cdot (I_{\text{max}} - i)}{I_{\text{max}}} \quad (7)$$

$$c_2^i = c_{\text{min}} + \frac{(c_{\text{max}} - c_{\text{min}}) \cdot i}{I_{\text{max}}} \quad (8)$$

5. Particle position and velocity update

The velocity of a particle determined the direction and magnitude of task assignment adjustments. In this scheme, velocity values were restricted to integers. The updated particle position was bounded: (1) if $p_i^{t+1} < 1$, we took the value of 1; (2) if $p_i^{t+1} > N$, we took the value of N .
a) Position update (9)

$$p_i^{t+1} = p_i^t + [v_i^{t+1}] \quad (9)$$

b) Velocity update (10)

$$v_i^{t+1} = w^i \cdot v_i^t + c_1^i \cdot \gamma_1 \cdot (p_{best_i} - p_i^t) + c_2^i \cdot \gamma_2 \cdot (g_{best} - p_i^t) \quad (10)$$

5. Design of decentralized auditing protocol for fog computing

5.1. Security objectives

Based on the dynamic group management strategy and task allocation optimization strategy, we design a secure and efficient data integrity auditing scheme for IoT environments. This scheme met the following five security objectives:

1. Correctness: (1) Only fog nodes with legitimate identities and resource values that met the required conditions could pass the VRF verification, ensuring the security and reliability of the MF election phase; (2) CSP could pass the integrity verification of OFs only when it stored data correctly.
2. Audit robustness: No adversary could launch forgery, replacement, or replay attacks with non-negligible probability, ensuring the security of the audit process.

3. Privacy protection: User data privacy would not be disclosed to other entities, and verifiers could not derive or reconstruct sensitive information from task execution.
4. Fog-Layer mechanism resilience: (1) The VRF-based election maintained its unpredictability and public verifiability under real-world conditions; (2) the ADPSO-based scheduler could resist orchestrated load-imbalance and input-manipulation attacks; (3) the system preserved audit integrity even in the presence of a limited number of malicious or colluding OF nodes.
5. Efficient decentralization mechanism: Through the consistent hash ring mechanism, the scheme supported the efficient dynamic joining and exiting of MFs, enabling rapid localized migration of auditing tasks.

5.2. Decentralized audit protocol process

As shown in Fig. 7, our scheme was divided into four phases: initialization, storage, audit task allocation, and auditing. Specifically:

- 1) Initialization phase: DO generated system parameters and key pairs, and employed a dynamic group management strategy to form a set of MF nodes. Firstly, DO divided fog nodes at the edge layer into multiple groups and elected an MF for each group. Secondly, the MF-related information was mapped onto a hash ring. Thirdly, each MF initialized the parameters related to the ADPSO algorithm.
- 2) Storage phase: DO encrypted the original file, split the encrypted file into multiple data blocks, and generated the corresponding block tags using the ZSS signature. These blocks were then uploaded to the CSP for storage. Meanwhile, each MF stored resource information and the geo-location directory of OFs within its group.

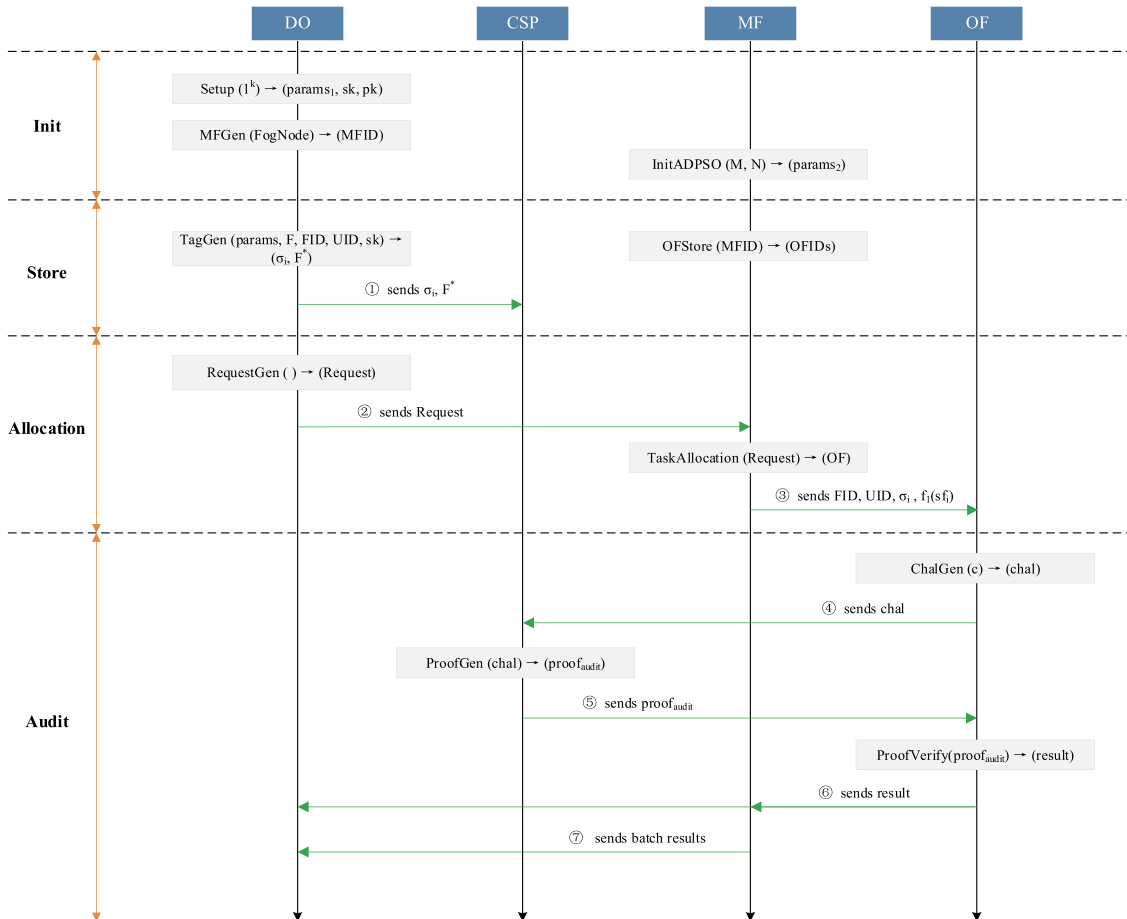


Fig. 7. Decentralized audit protocol process.

- 3) Audit task allocation phase: DO initiated an audit request to MF through a consistent hash ring. Upon receiving the request, MF utilized the ADPSO algorithm to allocate the audit task to the most suitable OF node based on the resource information of the OF nodes.
- 4) Audit phase: Upon receiving the audit task, OF initiated an audit challenge to the CSP. CSP could verify the challenge and return the corresponding integrity proof. OF then verified the proof and sent the audit result to either the MF or the DO.

5.2.1. Detailed algorithm description of the initialization phase

The initialization phase consisted of three algorithms: DO generated the system parameters, DO generated the set of fog nodes, and MF initialized the ADPSO algorithm parameters. These were described as follows:

- 1) Setup (1^k) $\rightarrow$ (params₁, sk, pk). DO selected a q-order additive cyclic group G_1 and a q-order multiplicative cyclic group G_2 , where g was the generator of G_1 , and constructed the bilinear pairing $e : G_1 \times G_1 \rightarrow G_2$. DO then chose the hash function $H_1 : \{0,1\}^* \rightarrow Z_p^*$ and the pseudorandom function $f_1 : Z_p^{**}\{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$. Subsequently, DO randomly selected an element x from Z_p^* and computed the public key $pk = x \bullet g$. DO published the params₁ = { G_1 , G_2 , g , q , H_1 , f_1 , e } and the public key pk .
- 2) MFGen (FogNode) $\rightarrow$ (MFID). The algorithm selected MFs based on a dynamic group strategy, obtained the MFID of each group, and mapped the key values of all MF nodes to a virtual hash ring.
- 3) InitADPSO (M, N) $\rightarrow$ (params₂). MF initialized the inertia weight coefficients, cognitive factors, and social factors based on the current number of audit tasks and the number of OFs within the group. It also set the initial positions and velocities of the particles, initialized the g_{pest} and p_{best} , and obtained params₂ = { w , c_1 , c_2 , v_0 , x_0 , g_{pest} , p_{best} }.

5.2.2. Detailed algorithm description of the storage phase

The storage phase consisted of two algorithms: DO generated the tags and MF generated the OF resource tables. These were described as follows:

- 1) TagGen (params₁, F, FID, UID, sk) $\rightarrow$ (σ_i , F*). DO encrypted the original file F using the AES symmetric encryption algorithm, producing the encrypted file F^* . The encrypted file F^* was partitioned into n large data blocks $\{m_{ij}\}$ according to the size, where i denoted the file index ($1 \leq i \leq m$), and j denoted the data block index ($1 \leq j \leq n$). DO generated the data block tag σ_{ij} for each data block m_{ij} , which combined the information such as the file identifier FID, the DO identifier UID and the file priority score sf_i . DO then sent $\{\sigma_i, F^*\}$ to the CSP for storage, where $\sigma_i = \{\sigma_{i1}, \sigma_{i2}, \dots, \sigma_{in}\}$

$$\sigma_{ij} = \frac{1}{H_1(m_{ij}) + x + H_1(FID \parallel UID \parallel j \parallel f_1(sf_i))} \cdot g \quad (11)$$

- 2) OFStore (MFID) $\rightarrow$ (OFIDs). MF stored the resource information and geo-location directories of OFs within its group for subsequent audit task allocations.

5.2.3. Detailed algorithm description of the audit task allocation phase

The audit task allocation phase consisted of two algorithms: DO initiated the audit request and MF allocated the audit task. These were described as follows:

- 1) RequestGen () $\rightarrow$ (Request). The DO located a suitable MF using a consistent hashing algorithm and initiated an audit request to that MF. The DO then sent the request = {FID, UID, σ_i , $f_1(sf_i)$, { ra_i , rc_i , rb_i , rs_i }} to the selected MF.

- 2) TaskAllocation (Request) $\rightarrow$ (OF). Firstly, MF obtained the resource information $\{c_i, b_i, s_i\}$ of the OF nodes in the current group. Secondly, MF utilized the ADPSO algorithm within the task allocation optimization strategy to allocate the auditing task to the most appropriate OF.

5.2.4. Detailed algorithm description of the audit phase

The audit phase consisted of three algorithms: OF initiated the audit challenge, CSP generated the proof, and the OF performed the integrity audit. These were described as follows:

- 1) ChalGen (c) $\rightarrow$ (chal). Upon receiving {FID, UID, σ_i , $f_1(sf_i)$ } from the MF, OF initiated an audit challenge to CSP. OF selected c numbers from the set $\{1, 2, \dots, n\}$ to generate the subset $I = \{s_1, s_2, \dots, s_c\}$, where n represented the number of data blocks, and each value corresponded to a block index. For each element j ($s_1 \leq j \leq s_c$) in subset I , OF generated $v_j \in Z_p^*$ and formed the chal = $\{j, v_j\}_{j \in I}$. The chal was then sent to CSP. OF could precompute eq. (12) and eq. (13) in advance for subsequent verification phases.

$$vpt_1 = \sum_{j \in I} v_j H_1(FID \parallel UID \parallel j \parallel f_1(sf_i)) \cdot g \quad (12)$$

$$vpt_2 = \sum_{j \in I} v_j \cdot pk \quad (13)$$

- 2) ProofGen (chal) $\rightarrow$ (proof_{audit}). The CSP received the chal and verified the validity of the challenge block indices. It then computed the data block and tag proofs μ (14) and δ (15) corresponding to each j in chal, and sent proof_{audit} = { δ , μ } to OF for verification.

$$\mu = \sum_{j \in I} v_j H_1(m_{ij}) \bullet g \quad (14)$$

$$\delta = g - \sum_{j \in I} \frac{v_j}{\sigma_{ij}} g^2 \quad (15)$$

- 3) ProofVerify (proof_{audit}) $\rightarrow$ (result). OFs received proof_{audit} and performed an integrity audit. They verified $e(\delta, g)e(\mu + vpt_1 + vpt_2, g) \stackrel{?}{=} e(g, g)$, and notified the DO or MF of the verification result.

5.3. Security analysis

5.3.1. Threat model

This paper considered the following threat models: (1) CSP was a semi-trusted entity that may suffer from insider attacks, software vulnerabilities, and storage device failure, leading to data loss or corruption. Some CSPs might maliciously delete or tamper with stored data to reduce storage costs, potentially launching forgery, replay, or replacement attacks. (2) OFs were honest and curious entities with respect to user data privacy, while a subset of OFs might become actively malicious or collude with each other to provide false audit results. (3) Despite MFs' proximity to the data source and strong computational and scheduling capabilities, MFs were also semi-trusted entities and remained curious about user data. An adversary might attempt to influence the VRF-based election or flood a specific MF with carefully crafted audit requests to disrupt scheduling optimality and induce load imbalance.

5.3.2. Correctness analysis

1. Only fog nodes with legitimate identities and resource values that met the required conditions could be verified by the VRF to ensure that the MF election phase was secure and reliable. The equation (16) validation held.

$$e(g^{\text{seed}} f p k i, \text{proof}) = e(g^{\text{seed}} g^{\text{fski}}, g^{1/\text{fski}+\text{seed}}) = e(g, g) \quad (16)$$

2. CSP could pass the OF's integrity verification only if it stored the

data correctly. Given $H_s = H_1(FID \parallel UID \parallel j \parallel f_1(sf_i))$, the following equation (17) validation held.

$$\begin{aligned}
 & e(\delta, g)e(\mu + vpt_1 + vpt_2, g) \\
 &= e\left(g - \sum_{j \in I} \frac{v_j}{\sigma_j} \cdot g^2, g\right) e\left(\sum_{j \in I} v_j H_1(m_{ij}) \cdot g + \sum_{j \in I} v_j H_s \cdot g + \sum_{j \in I} v_j \cdot pk, g\right) \\
 &= e\left(g - \sum_{j \in I} v_j \left((H_1(m_{ij}) + x + H_s) \cdot \frac{1}{g}\right) \cdot g^2, g\right) e\left(\sum_{j \in I} v_j H_1(m_{ij}) \cdot g + \sum_{j \in I} v_j H_s \cdot g + \sum_{j \in I} v_j \cdot pk, g\right) \\
 &= e\left(g - \sum_{j \in I} v_j ((H_1(m_{ij}) + x + H_s) \cdot g), g\right) e\left(\sum_{j \in I} v_j H_1(m_{ij}) \cdot g + \sum_{j \in I} v_j H_s \cdot g + \sum_{j \in I} v_j x \cdot g, g\right) \\
 &= e(g, g)
 \end{aligned} \tag{17}$$

5.3.3. Audit robustness

Theorem 1 No adversary could launch a forgery attack with non-negligible probability.

Proof Suppose an adversary tampered with the data m_{ij} to obtain a new data block m_{ij}^* . In that case, the adversary needed to construct a new data block signature $\sigma_{ij}^* = \frac{1}{H_1(m_{ij}^*) + x + H_1(FID \parallel UID \parallel j \parallel f_1(sf_i))} \cdot g$, which resulted in equation (18), equation (19), equation (20).

$$\mu^* = \sum_{j \in I \mid j \neq j} v_j H_1(m_{ij}) \cdot g + v_j H_1(m_{ij}^*) \cdot g \tag{18}$$

$$\delta^* = g - \sum_{j \in I \mid j \neq j} \frac{v_j}{\sigma_j} \cdot g^2 - \frac{v_j}{\sigma_{ij}^*} \cdot g^2 \tag{19}$$

$$e(\delta^*, g)e(\mu^* + vpt_1 + vpt_2, g) = e(g, g) \tag{20}$$

However, in this scheme, the adversary could not obtain the private key x . Since the probability of correctly guessing x was negligible, the adversary couldn't forge m_{ij}^* in a way that satisfied the equation $\sigma_{ij}^* = \sigma_{ij}$. Therefore, the scheme could resist forgery attacks.

Theorem 2 No adversary could initiate a replay attack with a non-negligible probability.

Proof OF sent $chal = \{j, v_j\}_{j \in I}$ to CSP. If the CSP attempted to reuse previously verified evidence $proof^* = \{\delta^*, \mu^*\}$, it would be invalid due to the presence of a random number $v_j^* \in Z_p^*$. If $proof^* = proof$ successfully passed the OF's verification, then v_j^* must equal v_j , which contradicted the assumption. Therefore, the scheme was resistant to replay attacks.

Theorem 3 No adversary could launch a replacement attack with non-negligible probability.

Proof This proof was similar to the proofs of Theorem 1 and Theorem 2.

5.3.4. Privacy protection

User data privacy was not leaked to other entities. Even if an adversary obtained σ_{ij} , they could not derive the specific content of m_{ij} , as $H(m_{ij})$ was computed using a collision-resistant hash function. Consequently, the verifier could not derive or reconstruct sensitive information during the audit execution process.

5.3.5. Fog-Layer mechanism resilience

1. Resilience of the VRF-based MF election

The VRF ensured the unpredictability and public verifiability of the MF election. In practical environments, its security depended on the

management of the private key fsk_i and seed. In our scheme, a new random seed was generated and broadcast in each election round, preventing adversaries from predicting or forging election outcomes. In

addition, the public verifiability of the VRF proofs allowed any node to verify the legitimacy of the elected MF using the published seed and public key, without relying on trust in other entities.

2. Robustness of the ADPSO-based task allocation

The ADPSO algorithm ran on the MF and could resist orchestrated load-imbalance and input-manipulation attacks. Even if a malicious MF attempted to bias task allocation by falsifying local information or tuning optimization parameters, the allocation results remained verifiable to other fog nodes, allowing abnormal scheduling behavior to be detected.

In addition, the discrete search space and iterative optimization process of ADPSO limited the influence of individual abnormal inputs on global allocation results. Combined with the load evaluation model and fitness function, audit tasks were distributed across multiple OFs, preventing adversaries from exhausting MF resources through fabricated high-priority audit requests.

3. Resistance to OF Collusion Attacks

In CAIFC, audit tasks were distributed across multiple OF nodes and validated through collaborative verification. Malicious behavior by a single OF could be detected through cross-checking or MF verification. In the case of OF collusion, randomized task allocation based on VRF and consistent hashing prevented adversaries from persistently controlling a sufficient number of audit tasks, thereby preserving audit integrity.

5.3.6. Efficient decentralization

This scheme adopted a consistent hash ring mechanism with virtual nodes to achieve efficient decentralized auditing, eliminating single-point dependencies and mitigating risks associated with single-point failures or tampering. Compared to blockchain technology, the mechanism eliminated consensus overheads and avoided the high storage costs of a global ledger.

In an ideal case, assuming the system had N MF nodes and each MF corresponded to V virtual nodes, the total number of nodes on the hash ring could be NV . If the number of user audit requests was M , then the average number of requests handled by each virtual node was: $u_{\text{virtual}} = M/(NV)$. The total load on each MF node was determined by the combined load of all its virtual nodes: $u_{\text{physical}} = V * u_{\text{virtual}} = M/N$. The consistent hashing mechanism had a good load-balancing property so that only the affected node and its neighboring nodes required task redistribution when adding or removing nodes. The average complexity of task migration in this mechanism was $O(M/N)$, ensuring high scalability and making it well-suited for dynamic node changes.

Table 2
Comparative analysis.

Schemes	Public verification	Decentralization	Privacy protection	Task-on-demand allocation	IoT Applicability
Wang et al. (2010)	✓	×	×	×	×
Chen et al. (2018)	✓	✓	×	×	✓
Tian et al. (2019)	✓	×	✓	×	✓
Zhao et al. (2022)	✓	×	✓	×	×
Chen et al. (2022)	✓	✓	✓	×	×
Bian et al. (2022)	✓	×	✓	×	✓
Ours	✓	✓	✓	✓	✓

5.4. Evaluation of decentralized audit protocols

5.4.1. Comparative analysis

As shown in Table 2, we conducted a comparative analysis of our scheme and the existing mainstream research schemes in five key aspects: public verification, decentralization, privacy protection, task-on-demand allocation, and IoT applicability. Firstly, all the schemes supported public verification, which allowed third parties to verify the correctness of audit results. However, the schemes proposed by Wang et al. (2010), Tian et al. (2019), Zhao et al. (2022), and Bian et al. (2022) adopted centralized auditing architecture, which did not support decentralization and was prone to single point of failures. Compared to Wang et al. (2010) and Chen et al. (2018), our scheme implemented a privacy-preserving mechanism that effectively prevented data leakage during the auditing process. Additionally, the schemes of Wang et al. (2010), Zhao et al. (2022), and Chen et al. (2022) relied solely on traditional cloud storage architectures, which were not well-suited for resource-constrained IoT environments and resulted in high computational and communication costs. Furthermore, none of the existing schemes involved on-demand audit task allocation. In contrast, our scheme introduced a task-on-demand allocation mechanism, which could achieve optimal allocation according to users' auditing requirements, and improve the flexibility and auditing efficiency of the system.

5.4.2. Performance comparison

This scheme focused on analyzing the computational overheads of the TagGen, ProofGen, and ProofVerify algorithms. We also considered the time overhead T_{ia} introduced by the audit task allocation in our ProofVerify algorithm. The computational overhead was measured in terms of the following operations: (1) multiplication T_m , (2) hash computation T_h , (3) exponential operation T_e , (4) inverse operation T_i , (5) bilinear pairing T_p . We assumed that a file was divided into n data blocks, each containing one segment, with c challenge blocks and M audit requests. In our scheme, the average number of requests allocated to each OF node was $\lambda \in [1, M]$. Additionally, in the scheme (Tian et al., (2019)), t denoted the number of data blocks generated by fog nodes and w represented the number of mobile sinks. Table 3 demonstrated the computational overheads of different schemes.

The communication overhead mainly considered the overhead of the DO sending information such as data blocks and tags to the CSP, and the

Table 3
Comparison of computational overheads.

Schemes	TagGen	ProofGen	ProofVerify
Wang et al. (2010)	$(2n+1)T_e + (n+1)T_{h-G_1} + nT_{m-G_1}$	$cMT_e + cMT_{m-z_p^*} + cMT_{m-G_1}$	$(c+1)MT_e + (c+1)MT_{m-G_1} + 2MT_p$
Zhao et al. (2022)	$(2n+5)T_e + (n+1)T_{h-G_1} + (3n+1)T_{m-G_1}$	$(c+3)MT_e + cMT_{m-z_p^*} + (c+1)MT_{m-G_1}$	$(2c+1)MT_e + (2c+1)MT_{m-G_1} + 4MT_p$
Tian et al. (2019)	$(5n+2t)T_e + nT_{h-G_1} + (n+t)T_{h-z_p^*} + (n+t)T_{m-G_1}$	$cMT_e + cMT_{m-z_p^*} + cMT_{m-G_1} + (2c+w)MT_p + MT_{h-z_p^*}$	$2cMT_e + (c+1)MT_{h-z_p^*} + cMT_p$
Bian et al. (2022)	$2nT_e + nT_{h-G_1} + nT_{m-G_1}$	$cMT_e + cMT_{m-z_p^*} + cMT_{m-G_1}$	$(c+1)MT_e + (c+1)MT_{m-G_1} + 3MT_p$
Ours	$nT_i + 2nT_{h-z_p^*} + nT_{m-G_1}$	$cMT_i + cMT_{h-z_p^*} + cMT_{m-z_p^*} + 2cMT_{m-G_1}$	$c\lambda T_{h-z_p^*} + c\lambda T_{m-z_p^*} + 2c\lambda T_{m-G_1} + 3\lambda T_p + T_{ia}$

Table 4
Comparison of communication overheads.

Schemes	DO $\rightarrow$ CSP	CSP $\rightarrow$ TPA
Wang et al. (2010)	$n Z_p^* + t + (n+1) G_1 $	$ Z_p^* + (2+c+\xi) G_1 $
Zhao et al. (2022)	$n Z_p^* + Fid + T_{fid} + (n+2) G_1 $	$ Z_p^* + T_{fid} + 4 G_1 $
Tian et al. (2019)	$n Z_p^* + 2n G_1 $	$ Z_p^* $
Bian et al. (2022)	$n Z_p^* + n G_1 - \tau n(Z_p^* + G_1)$	$ Z_p^* + G_1 - \tau(Z_p^* + G_1)$
Ours	$n Z_p^* + n G_1 $	$2 G_1 $

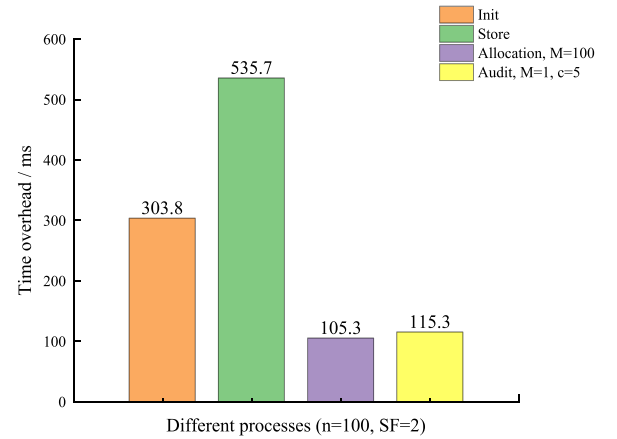


Fig. 8. The time overheads of four phases.

CSP sending proof to the verifier. $|*|$ denoted the size of an element in a given group, the length of an integer in Z_p^* , etc. ξ represented the auxiliary information in the scheme (Wang et al, 2010) and τ denoted the communication overhead function saved in the scheme (Bian et al., 2022). Table 4 presented the communication overheads of different schemes.

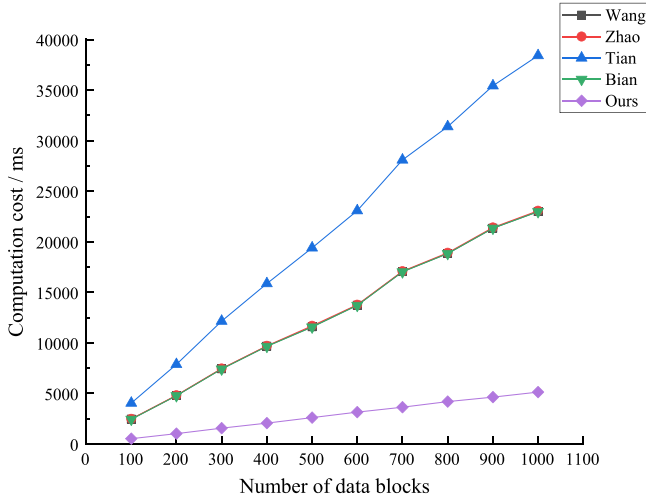


Fig. 9. Computational overhead of the TagGen algorithm.

5.4.3. Experimental results

The experiment was conducted on a Windows 10 system with the following hardware configuration: an Intel(R) Core (TM) i7-12700K processor (3.61GHz CPU) and 32GB RAM. The experiment was developed in Java language and implemented in the IntelliJ IDEA programming environment with Java pairing-based cryptography (JPBC) library. The experimental data were analyzed based on the average of 10 test results.

Our auditing scheme was divided into four phases: initialization, storage, audit task allocation and audit phase. Fig. 8 illustrated the time overhead of each phase when the system consisted of two groups, each containing 5 OF nodes, with the number of challenge blocks set to $c = 5$ and the number of data blocks set to $n = 100$.

Fig. 9 illustrated the computational overheads of different schemes in the TagGen algorithm when the number of data blocks increased linearly from 100 to 1,000. The schemes proposed by Wang et al. (2010), Zhao et al. (2022), and Bian et al. (2022) adopted the BLS signature to generate authentication tags, resulting in similar computational overheads. The scheme of Tian et al. (2019) had higher computational overhead than other schemes due to the need for secondary tag generation. Our scheme adopted a lightweight ZSS signature, which was more suitable for IoT environments, and demonstrated significantly lower computational overhead compared to other schemes.

Fig. 10 showed the computational overhead of different schemes in the ProofGen algorithm when the number of challenge blocks increased linearly from 5 to 50, with the audit task requests set to $M = 10$. When the number of challenge blocks increased to 50, the actual overhead of our scheme was only 5.165s with 10 audit requests initiated by the client. The computational overhead of this algorithm was slightly higher than that of the other schemes because privacy protection was considered in our scheme.

Fig. 11 represented the computational overhead of different schemes in the ProofVerify algorithm when the number of challenge blocks increased linearly from 5 to 50, with $M = 10$ audit task requests. Our scheme utilized a task allocation strategy to optimize the allocation of audit tasks so that they are reasonably distributed among multiple OF nodes. The observed reduction in verification latency indicated that audit tasks were effectively matched to fog nodes whose resource profiles were well aligned with task requirements. Moreover, the fitness function of the ADPSO algorithm ensured that no single OF became a performance bottleneck, thereby contributing to low verification latency. When the number of challenge blocks reached 50, the actual overhead was only 1.2118s, which was significantly lower than that of the other schemes.

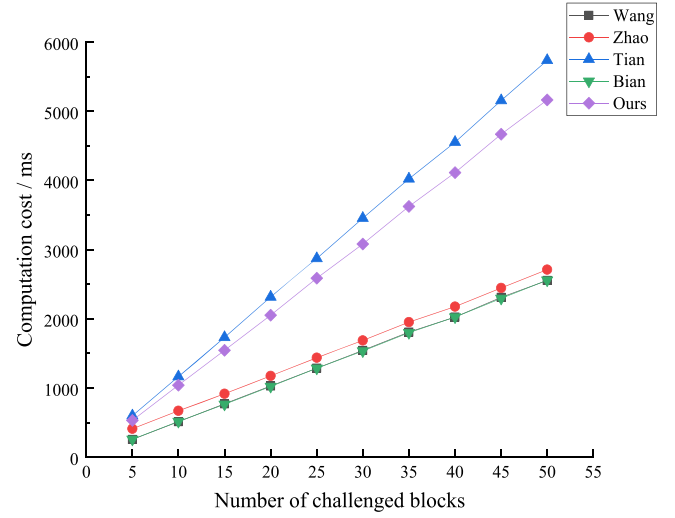


Fig. 10. Computational overhead of the ProofGen algorithm.

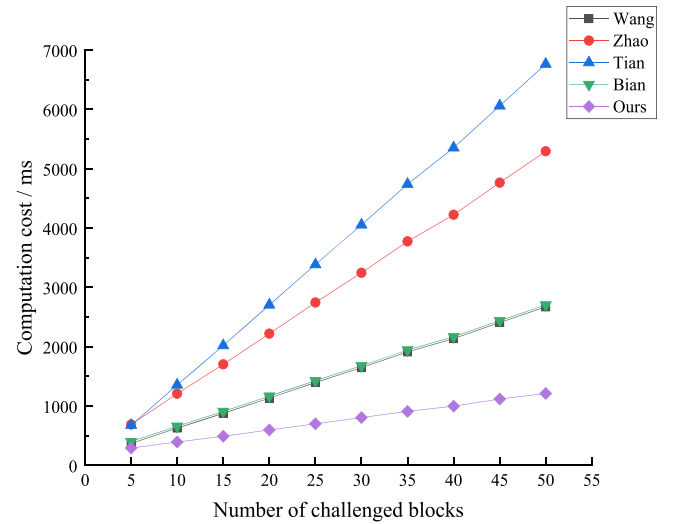


Fig. 11. Computational overhead of the ProofVerify algorithm.

6. Conclusion

In this paper, we proposed a collaborative audit scheme of IoT data integrity for fog computing, aiming to improve auditing efficiency and stability in IoT environments. Firstly, we proposed a dynamic group management strategy for fog nodes at the edge layer and formed a decentralized auditing architecture. The resource capabilities of fog nodes and VRF were utilized in the strategy, which could achieve the parallel execution of audit requests and reduce the audit response time from $O(n)$ in the traditional TPA-based scheme to $O(1)$ (n denoted the number of concurrent tasks). A two-layer hash mapping model with a consistent hash ring mechanism was integrated into the audit architecture. In this architecture, we mapped the key of data blocks and MFs onto a virtual ring, which not only facilitated the logical partitioning of audit tasks but also supported localized task migration during dynamic MF join /exit events. Secondly, we incorporated a multi-dimensional load evaluation model that evaluated task security levels and latency requirements, enabling hierarchical task processing and on-demand allocation. To achieve optimized auditing task allocation, we designed the ADPSO algorithm based on the model. The algorithm dynamically matched audit tasks to the most suitable fog nodes by simultaneously optimizing for latency, edge-layer contribution, and load balance.

Experimental results demonstrated that the proposed scheme reduced the audit latency effectively and improved the audit efficiency by at least 50.48% when the audit request reached 10 times. In conclusion, our scheme provided an efficient, low-latency, decentralized solution for data integrity auditing in IoT environments.

In the future, we plan to incorporate reinforcement learning to enable intelligent dynamic task allocation. Additionally, we will optimize the consistency hash ring mechanism to strengthen resilience against malicious node attacks and improve data security during task migration, ensuring end-to-end auditing integrity and reliability. Beyond algorithmic improvements, we also intend to extend the evaluation of CAIFC to large-scale, real-world IoT deployments. Such experiments will consider practical networking challenges, including varying bandwidth, intermittent connectivity, and dynamic workload fluctuations. Furthermore, comprehensive energy usage of fog nodes and IoT devices will be conducted to assess and optimize the energy efficiency of CAIFC in resource-constrained environments.

CRedit authorship contribution statement

Xinfeng He: Writing – review & editing, Supervision, Project administration, Funding acquisition. **Qing Zhou:** Writing – original draft, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the Innovation Capacity Enhancement Program-Science and Technology Platform Project of Hebei Province (22567638H)

Data availability

No data was used for the research described in the article.

References

- Ali, O., Ishak, M K, Bhatti, M K L, et al., 2022. A comprehensive review of internet of things: technology stack, middlewares, and fog/edge computing interface[J]. *Sensors* 22 (3), 995.
- Ateniese, G., Burns, R., Curtmola, R., et al., 2007. Provable data possession at untrusted stores. In: *Proceedings of the 14th ACM conference on Computer and communications security*, pp. 598–609.
- Bian, G., Fu, Y., Shao, B., et al., 2022. Data integrity audit based on data blinding for cloud and fog environment[J]. *IEEE Access*. 10, 39743–39751.
- Bonomi, F., Milito, R., Zhu, J., et al., 2012. Fog computing and its role in the internet of things. In: *Proceedings of the first edition of the MCC workshop on Mobile cloud computing*, pp. 13–16.
- Chang, J., Hao, X., 2023. Certificateless dynamic data sharing scheme with file recommendation and integrity auditing functions in cloud-fog environment[J]. *IEEE Internet. Things. J.* 11 (6), 9630–9644.
- Chataut, R., Phoummalayvane, A., Akl, R., 2023. Unleashing the power of IoT: A comprehensive review of IoT applications and future prospects in healthcare, agriculture, smart homes, smart cities, and industry 4.0[J]. *Sensors* 23 (16), 7194.
- Chen, Y J, Wang, L C, Wang, S., 2018. Stochastic blockchain for IoT data integrity[J]. *IEEE Trans. Netw. Sci. Eng.* 7 (1), 373–384.
- Chen, L., Fu, Q., Mu, Y., et al., 2022. Blockchain-based random auditor committee for integrity verification[J]. *Future Gener. Comput. Syst.* 131, 183–193.
- Gao, X., Yu, J., Chang, Y., et al., 2021. Checking only when it is necessary: Enabling integrity auditing based on the keyword with sensitive information privacy for encrypted cloud data[J]. *IEEE Trans. Dependable Secure Comput.* 19 (6), 3774–3789.
- Gong, T., Zhu, L., Yu, F R, et al., 2023. Edge intelligence in intelligent transportation systems: a survey[J]. *IEEE Trans. Intell. Transp. Syst.* 24 (9), 8919–8944.
- Gu, K., Wang, Y., Qiu, J., et al., 2024. Blockchain-based data deduplication and distributed audit for shared data in cloud-fog computing-Based VANETs[J]. *IEEE Trans. Netw. Serv. Manag.* 21 (5), 5548–5565.
- Han, H., Fei, S., Yan, Z., et al., 2022. A survey on blockchain-based integrity auditing for cloud data[J]. *Digit. Commun. Netw.* 8 (5), 591–603.
- Hu, J., Huang, K., Bian, G., et al., 2024. Redact4Trace: a solution for auditing the data and tracing the users in the redactable blockchain[J]. *Comput. Netw.* 245, 110360.
- Karger, D., Lehman, E., Leighton, T., et al., 1997. Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the world wide web. In: *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pp. 654–663.
- Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: *Proceedings of the IEEE International Conference on Neural Networks*, 4. IEEE, pp. 1942–1948.
- Liu, Z., Wang, S., Liu, Y., 2023. Blockchain-based integrity auditing for shared data in cloud storage with file prediction[J]. *Comput. Netw.* 236, 110040.
- Liu, D., Ding, Y., Yu, G., et al., 2024. Privacy-preserving dynamic auditing for regenerating code-based storage in cloud-fog-assisted IIoT[J]. *Internet Things* 25, 101084.
- Micali, S., Rabin, M., Vadhan, S., 1999. Verifiable random functions[C]// 40th annual symposium on foundations of computer science (cat. no. 99CB37039). IEEE 120–130.
- Sellami, Y., Imine, Y., Gallais, A., 2024. A verifiable data integrity scheme for distributed data sharing in fog computing architecture[J]. *Future Gener. Comput. Syst.* 150, 64–77.
- Singh, G., Chaturvedi, A K, 2024. A cost, time, energy-aware workflow scheduling using adaptive PSO algorithm in a cloud-fog environment[J]. *Computing* 106 (10), 3279–3308.
- Tian, H., Nan, F., Chang, C C, et al., 2019. Privacy-preserving public auditing for secure data storage in fog-to-cloud computing[J]. *J. Netw. Comput. Appl.* 127, 59–69.
- Tian, H., Wang, M., Quan, H., et al., 2025. TEEMRDA: Leveraging trusted execution environments for multi-replica data auditing in cloud storage[J]. *Comput. Secur.* 150, 104250.
- Wang, Q., Wang, C., Ren, K., et al., 2010. Enabling public auditability and data dynamics for storage security in cloud computing[J]. *IEEE Trans. Parallel Distrib. Syst.* 22 (5), 847–859.
- Wei, P C, Wang, D., Zhao, Y., et al., 2020. Blockchain data-based cloud data integrity protection mechanism[J]. *Future Gener. Comput. Syst.* 102, 902–911.
- Wen, J., Deng, L., 2024. Certificateless integrity auditing scheme for sensitive information protection in cloud storage[J]. *J. Syst. Archit.* 156, 103267.
- Xu, G., Bai, Y., Pan, Q., et al., 2017. Data verification tasks scheduling based on dynamic resource allocation in mobile big data storage[J]. *Comput. Netw.* 126, 246–255.
- Yan, H., Gui, W., 2021. Efficient identity-based public integrity auditing of shared data in cloud storage with user privacy preserving[J]. *IEEE Access*. 9, 45822–45831.
- Yang, K., Jia, X., 2012. An efficient and secure dynamic auditing protocol for data storage in cloud computing[J]. *IEEE Trans. Parallel Distrib. Syst.* 24 (9), 1717–1726.
- Yuan, H., Chen, X., Wang, J., et al., 2020. Blockchain-based public auditing and secure deduplication with fair arbitration[J]. *Inf. Sci.* 541, 409–425.
- Zhao, Y., Chang, J., 2022. Certificateless public auditing scheme with designated verifier and privacy-preserving property in cloud storage[J]. *Comput. Netw.* 216, 109270.
- Zhao, H., Yao, X., Zheng, X., et al., 2019. User stateless privacy-preserving TPA auditing scheme for cloud storage[J]. *J. Netw. Comput. Appl.* 129, 62–70.
- Zhou, H., Shen, W., Liu, J., 2025. Certificate-based multi-copy cloud storage auditing supporting data dynamics[J]. *Comput. Secur.* 148, 104096.